SOP: Building the ragcap multi-agent RAG system

Standard operating procedure. Reproduces the design in Kumar's RAG Capstone PDF (router, document RAG with grade/rewrite, SQL agent, general agent, pgvector, LangGraph, FastAPI, UI) on a 2019 Intel MacBook Pro (macOS 15.1.1) with local models. Written 2026-10-18.

1. Purpose, scope, result

Purpose: a repeatable procedure that a second engineer can follow to stand up the same system and reproduce every result. Scope: steps 1-20 are done and verified: the build (1-8), the improvement cycle (9-14), honest measurement with an unseen set and repeated runs (15-18), a recorded negative result (19) and version-aware retrieval with its OFF/ON proof (20). Still open: the P&ID vision branch, LangSmith, the Next.js front end; Docling and a stronger grader are blocked on this hardware (runbook section 6, step 19). The steps are in the order they happened, so early counts (8 documents) predate the versioned corpus of step 20 (16 documents).

Item	Value
Repository	github.com/shreenathacc22/ragcap: code at the root, these documents in docs/, every result in results/
History	first commit 6486837 is the golden freeze, before any pipeline code; one commit per step after that
Database	Postgres 16 + pgvector 0.8.7 in Docker container ragcap-pg, host port 5433 (localhost only)
LLM	Ollama llama3.2:3b through the OpenAI-compatible endpoint http://localhost:11434/v1
Embeddings	Ollama nomic-embed-text, 768 dimensions
Python	3.12 venv the .venv (psycopg, pgvector, openai, langgraph, pandas, fastapi, gradio)

2. Prerequisites

Requirement	Check command	Expected
Docker Desktop installed (daemon may be stopped)	open -a Docker; docker info	Server version printed (20.10.7 here)
Ollama running with two models	ollama list	llama3.2:3b and nomic-embed-text listed
Python 3.12	python --version in the venv	3.12.x
Homebrew is NOT installed on this Mac	which brew	not found: that is why Postgres runs in Docker

3. Procedure

Step 1. Database (Postgres + pgvector). Generate a random password into the repository's .env (mode 600, gitignored), pull the image, start the container, enable the extension.

```
cd ragcap
PW=$(openssl rand -hex 12)
printf 'PGHOST=localhost\nPGPORT=5433\nPGUSER=ragcap\nPGPASSWORD=%s\nPGDATABASE=ragcap\n' "$PW" > .env; chmod 600 .env
docker pull pgvector/pgvector:pg16
docker run -d --name ragcap-pg -e POSTGRES_USER=ragcap -e POSTGRES_PASSWORD="$PW" -e POSTGRES_DB=ragcap \
  -p 127.0.0.1:5433:5432 -v ragcap_pgdata:/var/lib/postgresql/data --restart unless-stopped pgvector/pgvector:pg16
docker exec ragcap-pg psql -U ragcap -d ragcap -c "CREATE EXTENSION IF NOT EXISTS vector;"
```

Why port 5433 and 127.0.0.1: avoids clashing with any local Postgres on 5432 and keeps the database off the network. Verify: extension version 0.8.7.

Step 1b. Python environment and schema.

```
python -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python step1_check.py      # applies schema.sql, stores 2 real embeddings, finds the right one, cleans up
```

schema.sql creates: **documents** (name, version, superseded, source_path), **chunks** (text, embedding vector(768), meta jsonb, HNSW cosine index), **uploaded_tables** (registry of CSV/Excel tables the SQL agent may touch). Pass criterion: prints 'STEP 1 OK'. If the embedding model changes, EMBED_DIM and the vector(768) column must change together and everything must be re-ingested.

Step 2. Ingestion. ragcap/ingest.py converts a file to Markdown, packs whole paragraphs into ~180-word chunks, prepends the document name before embedding, embeds in batches of 16 and writes chunks. CSV/Excel become real SQL tables (column names sanitised, types mapped to bigint/double/boolean/text, loaded with COPY) and are registered in uploaded_tables.

```
.venv/bin/python -m ragcap.ingest data/docs      # 8 Kafka runbooks -> 8 documents, 8 chunks
.venv/bin/python -m ragcap.ingest data/tables    # broker_inventory.csv -> table with 6 rows
docker exec ragcap-pg psql -U ragcap -d ragcap -c "select count(*) from chunks" -c "select * from uploaded_tables"
```

Pass criterion: counts match the input files. PDFs fall back to pypdf text extraction (Docling is blocked on this hardware, see Runbook section 6). .docx/.pptx/.html currently raise an error in this venv.

Step 3-4. LangGraph pipeline (ragcap/graph.py). State carries question, history, route, standalone question, chunks, relevant chunks, rewrite count, SQL, rows, answer and a per-node trace.

Node	What it does	LLM calls
router	JSON {route, standalone}; prompt lists the uploaded tables and their columns, documents-vs-database rules and unrelated examples; guard: route general is only accepted for greetings/thanks/self-questions (regex); standalone must copy the question word for word and only resolve pronouns (changed in steps 9, 13)	1
retrieve	embeds the standalone question, pgvector cosine search, top 6 chunks	0 (1 embedding)
grade	one yes/no JSON call PER retrieved passage (3 worker threads); keeps passages on the same topic as the question; inclusive prompt (changed in step 13)	6
rewrite	synonym/expanded query; allowed once (rewrites counter), then retrieve+grade again	1
generate	answer from numbered passages with [n] citations; if none relevant returns 'not in the knowledge base' without an LLM call	0-1
sql_agent	prompt = columns, 3 sample rows, allowed text values; validates single SELECT (no ';', forbidden keywords), runs in READ ONLY transaction with 5 s statement_timeout, one retry with the DB error	1-2
generate_sql	plain-language answer from the result rows; must copy numbers exactly, no unit conversion (step 10)	1
general	greetings/self questions; prompt forbids stating facts	1

```
.venv/bin/python try.py "At what consumer lag should we page someone?" "Which brokers are above 85% disk usage?"
```

Every run appends one line to traces.jsonl (route, per-node ms, calls, tokens, answer). LangSmith can be added later by setting its environment variables; it is not configured.

Step 5. Frozen evaluation set. golden/make_golden.py writes golden/golden.jsonl (27 cases): 16 document cases reused from the Track A B1 set (current-version facts only), 6 database, 3 general, 2 unanswerable. Fact strings accept alternatives with '|'. The file was committed (6486837) before any tuning.

Step 6. Metrics. eval.py runs every case through the graph and writes results/eval_<ts>.json and .md: router accuracy, answer correctness (by type), retrieval hit rate (all expected facts appear in the retrieved chunks), and a per-flow table of calls, tokens, latency and an equivalent cost at gpt-4.1-mini list prices ($0.40 / $1.60 per million tokens in/out) so it can be put next to Kumar's table. Actual cost is $0 (local).

```
.venv/bin/python eval.py          # all 27 cases, about 15-20 minutes on this CPU
.venv/bin/python eval.py 5        # first 5 cases only
```

Step 7. Tracing. ragcap/tracing.py (JSONL, see above).

Step 8. API and UI.

```
.venv/bin/uvicorn api:app --port 8000     # GET /health, POST /upload (multipart), POST /chat (SSE), POST /eval
curl -N -X POST localhost:8000/chat -H 'content-type: application/json' \
  -d '{"question":"How many brokers have status critical?"}'
.venv/bin/python ui.py                    # Gradio chat + upload, http://127.0.0.1:7860
```

/chat streams one 'node' event per graph node, then an 'answer' event with route, citations, SQL and total time. Verified: /health, /upload, /chat; ui.py imports cleanly (not opened in a browser).

4. Baseline results (2026-10-18, llama3.2:3b, 27 cases)

Metric	Result
Router accuracy	88.9% (24/27)
Answer correct overall	77.8% (21/27)
Retrieval hit rate (document cases)	81.3% (13/16)
By type	exact_string 6/6, version_current 5/8, multi_hop 1/2, sql 4/6, general 3/3, unanswerable 2/2
Document flow	15 answers, 3.0 LLM calls, 2044/66 tokens, 37.8 s, ~$0.00092 equivalent
Database flow	8 answers (incl. 2 misrouted documents questions), 3.0 calls, 465/76 tokens, 9.5 s, ~$0.00031
General flow	4 answers, 2.0 calls, 230/44 tokens, 4.1 s, ~$0.00016

Not comparable to Kumar's latency table: his is a hosted gpt-4.1-mini (about 2.5 s per question); this is a 3B model on an Intel CPU with Docker and a Docling build competing for CPU during the run (about 38 s). Compare quality columns, not latency.

5. Lessons learned while building

- The first grade prompt ('which passages help answer') made llama3.2:3b return [] for a question whose top hit was the right runbook, so the pipeline said 'not in the knowledge base'. Making the prompt inclusive fixed that case. A 3B grader is the weakest link; consider per-chunk grading or a larger model.

- Router misroutes (3/27): two document questions went to 'general' and 'database'. A single database table pulls 'disk threshold' questions toward SQL.

- SQL answers: the SQL is right but the wording step garbled results (total capacity reported as '16 GB' instead of 16000 GB; 'broker #3 with 1 topic' instead of broker 5 with 210). The generate_sql step should quote values from the rows verbatim.

- Two of the golden multi-source cases fail by mixing sources, the failure the B1 version filter is meant to address.

6. Improvement cycle: steps 9-14 (2026-10-18)

Goal: fix the three weak points in the baseline and re-measure on the SAME frozen golden set. Rule used: never edit golden/golden.jsonl; never put golden questions into prompts as examples (test leakage). Each step below was verified before the next began.

Step	Problem found (evidence)	Change	Verification
9 Router	3 document questions misrouted (2 to database, 1 to general). Router prompt did not say what the table holds; a 'disk threshold' question looked like SQL.	Router prompt now lists table names and columns, says a runbook rule is NOT in the tables, adds unrelated examples; added regex guard so 'general' is accepted only for greetings/thanks/self-questions. First draft copied golden questions as examples: caught and replaced before any eval.	7 probe questions routed 7/7 (3 previously wrong)

Step	Problem found (evidence)	Change	Verification
10 SQL	db-004 SQL wrong (GROUP BY broker_id gave count 1); db-003 SQL right but answer wrote '16 GB' for 16000.	SQL prompt rules: WHERE only if the question names the condition; never GROUP BY the id; LIMIT 1 only for a single extreme; SUM/AVG for totals. Answer prompt: copy numbers exactly, no units added. Three prompt iterations were needed: wording about 'WHERE' and 'LIMIT' leaked into unrelated queries on the 3B model.	6 db probes correct incl. broker 5 / 210 topics, 16,000 GB, 69.37%
11 Scorer	doc-008 marked wrong though answer 'factor ... is 3' was right: frozen fact was the literal 'replication factor 3'.	golden.jsonl left untouched. golden/amendments.jsonl (own commit ec7c609) sets facts to ['3'] with the reason; eval.py applies and prints 'amended doc-008'.	git log shows amendment separate from the frozen set
12 Run 2	-	Re-ran all 27 cases	router 100%, answers 92.6% (25/27), retrieval hit 100%, sql 6/6. New failures doc-009, doc-011 (exact strings).
13 Grader	doc-009/011 said 'not in the knowledge base' although retrieval found the right text. Tracing showed (a) router's standalone question had gained words 'What does a runbook say about...' and (b) the single-call grader kept unrelated passages and dropped the best one.	Router: standalone must be verbatim except pronouns. Grader: one yes/no call per passage instead of choosing indices from six.	4 probe questions: 3 correct passages kept, unanswerable kept none
14 Run 3	-	Re-ran all 27 cases	router 100%, answers 88.9% (24/27), retrieval hit 100%. Failures: doc-004 (grader dropped TLS host passage twice), doc-015 (multi-hop kept only one passage), db-004 (SQL grouped by rack).

Commands used (all from the repository root):

```
.venv/bin/python try.py "question 1" "question 2"           # one-off probes (route, path, SQL, answer)
.venv/bin/python - <<'EOF'                                  # stage-level probe, see Runbook section 7
EOF
git add golden/amendments.jsonl && git commit -m "Amend doc-008 fact string ..."      # amendment in its own commit
git add -A && git commit -m "Fix router prompt and guard, SQL prompt rules ..."
nohup .venv/bin/python eval.py > results/eval_run2.log 2>&1 &                      # full run in background
while ! grep -q '^saved' results/eval_run3.log; do sleep 15; done; grep BAD results/eval_run3.log
```

Results across runs (same 27 golden cases, llama3.2:3b, doc-008 amended from run 2 on):

Metric	Run 1 baseline	Run 2	Run 3	Target
Router accuracy	88.9% (24/27)	100% (27/27)	100% (27/27)	>= 95%
Answer correct	77.8% (21/27)	92.6% (25/27)	88.9% (24/27)	>= 90%
Retrieval hit rate (document)	81.3% (13/16)	100% (18/18)	100% (18/18)	>= 90%
version_current	5/8	8/8	7/8	-
exact_string	6/6	4/6	6/6	-
multi_hop	1/2	2/2	1/2	-
sql	4/6	6/6	5/6	6/6

Metric	Run 1 baseline	Run 2	Run 3	Target
unanswerable	2/2	2/2	2/2	2/2
Document flow LLM calls / latency	3.0 / 37.8 s (CPU contended)	3.0 / 29.1 s	8.0 / 25.4 s	-
Document+rewrite flow	none occurred	none occurred	3 answers, 14.3 calls, 41.5 s	-

Reading the table: router and retrieval improvements are real and stable (100% in both reruns). Answer correctness is 88.9-92.6% and the failing cases CHANGE between runs, which is model variability on a 3B model (different grader/SQL choices), not a single fixable bug. Quality is therefore reported as a range, not one number. Unanswerable questions now trigger the rewrite loop as designed (3 document+rewrite answers in run 3, matching Kumar's observation). The per-passage grader raises LLM calls from 3 to 8 for a document question: that is the cost of reliability here; against a hosted model a single-call grader would be preferable.

7. Measuring honestly: steps 15-17 (2026-10-18 evening)

After step 14 every one of the 27 golden cases had been looked at while fixing prompts, so any further score on that file would be partly memorised. Steps 15-17 add an unseen test set, repeated runs, and the hardening that a failed series forced.

Step	What	How	Verification
15 Held-out set	25 new cases written directly from the 8 runbooks and the broker table, on facts the first set never asked about (certificate validity, min.insync.replicas, casing rule, warn threshold, secondary contact...). 16 documents, 5 database, 2 general, 2 unanswerable.	golden/make_heldout.py -> golden/golden_heldout.jsonl; frozen in its own commit (83ea0cd) BEFORE the first run. Rule: never tune a prompt against these cases.	git log shows the freeze commit precedes any held-out result
16 Series harness	Same code, run 3x on the held-out set and 2x more on the original set, to report a mean and range instead of one lucky number.	eval.py gained --golden and --tag; run_series.sh <label> <model> runs the five evaluations in sequence in the background.	five result files per label under results/
17 Hardening	The first series died twice. (1) parse_json raised on truncated JSON from the 3B model (rewrite allowed only 60 output tokens; a long rewritten query was cut mid-string) and one bad case killed a 20-minute run. (2) Docker Desktop and Ollama had been stopped between runs, so later runs failed on their first call and wrote empty logs.	parse_json never raises (repairs a cut-off string or returns {}); chat() retries 3x with backoff on connection errors; eval.py records a failing case as an error row and continues; run_series.sh checks Postgres and Ollama before starting; rewrite/router token limits raised to 120/200. Failed logs kept in results/failed_series_20261018/.	parser unit test on 7 malformed inputs; 3-route smoke test; commit 9bbd8c4

```
.venv/bin/python golden/make_heldout.py                      # writes golden/golden_heldout.jsonl
git add golden/make_heldout.py golden/golden_heldout.jsonl && git commit -m "Freeze held-out golden set"
nohup ./run_series.sh llama3b llama3.2:3b > results/series_llama3b.out 2>&1 &     # ~1.5 h on this CPU
ls results/llama3b_*; cat results/llama3b_series.done        # exists only when all five runs finished
```

Step 18. Series results (llama3.2:3b, all nodes; 3 held-out runs + 4 original-set runs; 0 errors). Commit ff28602; full tables in results/llama3b_summary.md.

Metric	Original set (tuned on), mean of 4	Range	Held-out set (unseen), mean of 3	Range
Router accuracy	100%	100-100%	96.0%	96-96%
Answer correct	88.9%	85.2-92.6%	81.3%	80.0-84.0%
Retrieval hit rate (document)	100%	-	100%	-
Unanswerable refusals	100%	-	100%	-

Metric	Original set (tuned on), mean of 4	Range	Held-out set (unseen), mean of 3	Range
General	100%	-	100%	-
SQL	88%	83-100%	80%	80-80%
Document (version_current)	91%	88-100%	77%	75-81%

How to read it. (1) The 7.6-point gap between the tuned set (88.9%) and the unseen set (81.3%) is the honest measure of how much the prompt tuning in steps 9-14 fitted those 27 questions rather than the task. (2) Routing and retrieval generalise: 96-100% and 100% on unseen questions. (3) The held-out failures are deterministic, not noise: four cases failed in all three runs (h-doc-05, h-doc-08, h-doc-15, h-db-01) and one in two of three (h-doc-04). (4) Three of the four are the same mechanism: the right passage was retrieved, the 3B per-passage grader said no because the question was phrased differently from the runbook ('how many minutes must lag stay high' vs 'sustained for 5 minutes'), one rewrite, still nothing, answer 'not in the knowledge base'. On unseen questions a third of document answers went through the rewrite loop (25 of 75), which is the grader rejecting too often. (5) h-db-01 ('highest disk usage percentage and which broker') is a router ambiguity between a rule in a runbook and a measurement in a table. (6) Latency on this CPU: document 17-24 s, with rewrite 26-30 s, database 9-10 s, general 3-4 s. The grader is therefore the next thing to test, with a stronger model (step 19).

```
.venv/bin/python summarize_series.py llama3b        # mean / min / max, failing ids, per-flow table -> results/llama3b_summary.md
```

Step 19. Stronger grader trial: qwen3:8b (negative result, 2026-10-19 00:00-00:40)

Motivation: 3 of the 4 deterministic held-out failures were the 3B grader rejecting the right passage. Hypothesis: an 8B model grades better. Code: GRADE_MODEL / GRADE_MODE (per_passage | single | escalate) / GRADE_MAX_TOKENS / GRADE_PASSAGE_CHARS in ragcap/config.py; defaults reproduce the llama3b series exactly (commits de55379, 15414f7). 'escalate' = small model per passage first, big model single call only when nothing was kept, so the expensive call lands only on the hard cases.

Probe	Setting	Observed
1 passage, yes/no	qwen3:8b, JSON mode, max_tokens 200	Correct ('relevant: true' on the exact passage llama3b rejected 3/3). 52.7 s first call, 33.6 s warm. Thinking is kept in a separate 'reasoning' field; '/no_think' did not disable it (150-160 hidden tokens).
6 passages x 900 chars, single call	GRADE_MODE=single, max_tokens 600	No return after 20 min; killed. Python imports took >2 min because the CPU was saturated (load average 15 on 12 cores).
6 passages x 400 chars, single call	escalate path, max_tokens 400	No return after 10 min; killed. The qwen3 llama-server process had been at 578% CPU for 56 min continuously: Ollama keeps generating for a disconnected client, so the killed probe kept competing with the next one.

Conclusion: on a 2019 Intel CPU qwen3:8b can answer a short grading question in half a minute but cannot read six passages and reason about them in a usable time. The hypothesis is untested, not refuted: on a GPU or a hosted model the escalate design is the one to run (the switch is in place and costs nothing when unused). Recovery after the stuck runner: quit Ollama, kill any remaining llama-server, reopen Ollama, verify with a 'Thanks!' probe (runbook section 5).

```
osascript -e 'quit app "Ollama"'; sleep 3; pkill -f llama-server; open -a Ollama      # clear a stuck runner
# the trial to run on faster hardware:
GRADE_MODEL=qwen3:8b GRADE_MODE=escalate GRADE_MAX_TOKENS=400 N_HELD=1 N_ORIG=1 ./run_series.sh qwen3esc
```

Step 20. Version-aware retrieval: the filter and its OFF/ON proof (2026-10-19)

Why: Kumar's design (and ours until now) retrieves by similarity only. When a runbook exists in an old and a current version, the two read almost identically, so the old one is retrieved as often as the new one and the answer quotes a retired threshold, port or team. Method: Shree's B1 procedure (skill version-aware-rag), applied to pgvector instead of Chroma. The frozen golden sets already carry the trap: must_not_contain holds the stale values.

Sub-step	What was done	Verification
20a Corpus	The 16 Kafka runbooks from Track A (8 titles x v1/v2) ingested from data/docs_versioned/. Front-matter (doc, version, effective_date, superseded, status) is read as metadata and STRIPPED before embedding, so status never leaks into the vector. documents.name is the shared title; effective_date column added.	16 documents, 8 titles x 2; every current effective_date later than its old one (psql listing)
20b Data checks	b1_check.py: counts, exactly one current per title, integer versions, dates ordered, no front-matter or SUPERSEDED banner in any chunk, chunk meta agrees with the document. --negative breaks the data three ways in memory.	PASS; all three negative tests reported
20c The trap, measured	Pair cosine similarity old vs current with the pipeline's own embedder (nomic-embed-text); top-1 for the 8 plain version questions.	0.962-0.990 similarity; old version ranked first for 1/8 plain questions (weaker than Track B's 4/8 because each current runbook here also names its former value)
20d The filter	retrieve(): one condition, WHERE d.superseded = false, in the SQL before ORDER BY/LIMIT, so an old version never competes for a slot. Behind VERSION_AWARE (env, default OFF until this step was recorded) and ask(..., version_aware=). pgvector 0.8 hnsw.iterative_scan=relaxed_order so a filtered index scan still returns k.	commit 4d9c3b1 precedes every result file
20e Retrieval ablation	b1_eval.py: 32 frozen document cases (12 version-trap, 20 control) through retrieve() OFF then ON; no LLM, deterministic; interleaved OFF/ON latency, 3 rounds, 96 calls each, after a warm-up.	table below; results/version_filter_off.json, _on.json, ablation_summary.md
20f Answer-level ablation	eval.py (LLM in the loop) on the 27-case set with VERSION_AWARE=0 then 1, via run_answer_ablation.sh under caffeinate.	see the row added below when complete

Metric (retrieval only)	Filter OFF	Filter ON
Superseded chunks retrieved (headline)	99/192 (51.6%)	0/192 (0.0%)
Version-trap cases passing: facts present, no stale value from a superseded chunk	2/12	12/12
Version-trap cases with the stale value arriving from a superseded chunk	10/12	0/12
Strict: stale string anywhere in context (the current runbook quotes its own former value)	10/12	6/12
Control cases passing (must stay flat)	20/20	20/20
Median retrieve ms, interleaved x3, 96 calls each (fair)	56.0	57.0

Reading it: the filter removes every superseded chunk, fixes all 12 trap cases at retrieval level, leaves all 20 control cases untouched and costs 1 ms. The strict row stays at 6/12 ON because the current runbooks say things like 'the former 85% page threshold ... is superseded': the stale string is in context but from the right document. Whether the model then reports 75% and not 85% is an answer-level question (20f).

Step 20f. Answer-level OFF/ON (llama3.2:3b, the 27-case set, LLM in the loop, run_answer_ablation.sh under caffeinate; results/answer_off_*.json and answer_on_*.json):

Metric (answer level)	Filter OFF	Filter ON
Answer correct overall	24/27 (88.9%)	24/27 (88.9%)
version_current (8 cases)	8/8	8/8
multi_hop (2 cases)	0/2	1/2 (doc-016 recovered)
exact_string (6 cases)	6/6	5/6 (doc-011: the grader-variable case seen in run 2; retrieval hit was 100%)

Metric (answer level)	Filter OFF	Filter ON
sql / general / unanswerable	5/6, 3/3, 2/2	5/6, 3/3, 2/2 (db-004 fails both)
router accuracy, retrieval hit rate	100%, 100%	100%, 100%
Document flow latency (CPU state, not claimed)	26.6 s	19.3 s

Why 8/8 even with the filter OFF: every current runbook in this corpus retires its former value in its own text ('the previous 10,000-message threshold ... is no longer in force', 'the former 85% page threshold ... is superseded'). Shown both versions, the 3B model picked the current one every time. That is a property of this corpus (Track A's), not of the pipeline: the Track B corpus had no retirement sentences and produced stale answers (skill version-aware-rag, results table). What the answer-level run does establish: (1) multi-hop improved 0/2 to 1/2, because with the old versions excluded the top-6 holds more distinct documents instead of two copies of one topic; (2) nothing regressed because of the filter: doc-011 failed ON in the grader with retrieval hit 100%, the same variability seen in run 2. The retrieval ablation (20e) remains the proof the design needs: the filter is a rule in code that removes the dependency on the model reading carefully, at +1 ms.

Step 20g. Default flipped to ON. ragcap/config.py: VERSION_AWARE defaults to 1. OFF (VERSION_AWARE=0) remains the ablation setting and is what scenario V1 OFF in docs/RECREATE.md uses to show the trap. recreate.sh was run end to end after the flip and matched every fingerprint (counts, golden sha1s, corpus sha1, ablation headline).

```
.venv/bin/python -m ragcap.ingest data/docs_versioned          # 16 files; front-matter becomes metadata
.venv/bin/python b1_check.py --no-trap --negative              # structure + 3 injected faults -> PASS
.venv/bin/python b1_check.py                                   # + similarity trap (8 embedding calls)
caffeinate -i .venv/bin/python b1_eval.py --latency 3          # OFF/ON + fair latency, ~1 min
nohup ./run_answer_ablation.sh > results/answer_ablation.out 2>&1 &     # answer-level OFF then ON, ~40 min
VERSION_AWARE=1 .venv/bin/python try.py "At what disk utilisation should a broker page?"   # spot check
```

Step 21. Browser checks of both interfaces (2026-10-19). Until now the Gradio UI had only been imported. Headless Chromium (Playwright) opened each interface, typed a question, waited for the answer and took a screenshot (docs/img/). Gradio: 'At what disk utilisation should a broker page?' answered 75% with the trace line 'router 2302ms > retrieve 616ms > grade 17355ms > generate 4049ms'. Web page (step 23): 'Which brokers are above 85% disk usage?' showed three node pills as they finished (router 2133 ms, sql_agent 5471 ms, generate_sql 4664 ms), the SQL, and brokers 3 and 5.

Step 22. LangSmith, opt-in. ragcap/llm.py wraps the OpenAI client with langsmith.wrappers.wrap_openai when LANGSMITH_TRACING=true; LangGraph reports its node runs under the same variable, so LLM calls nest under nodes. Needed in .env: LANGSMITH_TRACING=true, LANGSMITH_API_KEY, LANGSMITH_PROJECT. Verified: off by default (no change); with a bogus key and a dead endpoint the question is still answered and the tracer only logs errors (non-fatal). Not verified: an actual upload, which needs a real API key. Own JSONL tracing (step 7) stays the default.

Step 23. Front end: Next.js descoped, a static page on the streaming API instead. Decision: Node 24 is installed, so Next.js was possible, but it adds a build toolchain and hosting without changing anything this project measures; the streaming contract (POST /chat as server-sent events, one event per graph node) is the design, and any front end consumes it. web/index.html (no build step) is served by FastAPI at / and shows node pills as they arrive, then the answer, route, SQL and citations, keeping the last three turns as history. Verified in Chromium (step 21). If Next.js is wanted later, it would call the same endpoint.

Step 24. P&ID multimodal branch (Kumar's proposed direction), scoped by evidence. Real P&IDs are confidential and GPU-class vision models are impossible on this Mac, so: (a) pid/make_pid.py draws three synthetic P&IDs from one template (tank, pump, heat exchanger, tank; two isolation valves, one control valve, five ISA-5.1 instruments, a legend and a title block), deterministic, with every tag and relation written to pid/ground_truth.json and 12 golden questions derived from it; (b) ragcap/vision.py asks a vision model to describe a drawing in one-fact-per-sentence form with a tag legend in the prompt, and ingest.py stores that description as the drawing's chunk with the image path in meta; (c) pid_eval.py scores tag extraction against ground truth and runs the 12 questions through the full graph. Probe: qwen2.5vl:3b (3.2 GB) read drawing 1 in 98 s on CPU with 11/13 tags, correct valve and

instrument relations, one invented tag (P-102) and two misses, so the branch was built and measured rather than descoped.

Run 1 (fixture as first drawn, instrument bubbles radius 32 px; results/pid_eval_run1_fixture_r32.log, drawings kept in results/pid_run1_fixture/):

Drawing	Tags expected	Correct	Invented	Missed	Vision seconds
P&ID-001	13	11	-	FCV-101, FIC-101	29.1 (warm)
P&ID-002	13	12	-	FIC-201	96.9
P&ID-003	13	11	-	FCV-301, FIC-301	78.7

QA, 12 frozen questions through the full graph (run 1)	Result	Cause found
Routed to documents / right drawing retrieved	100% / 100%	retrieval works on a vision description like on any document
Answer correct overall	4/12 (33%)	
what an instrument measures, on what	3/3	
valve isolating a pump	1/3	description says 'HV-101 between T-101 and P-101' and the drawing was retrieved, but the 3B grader rejected it because 'suction' is not in the text: the known grader weakness (steps 18-19), not the vision branch
controller + control valve	0/3	the 7-character tags FIC/FCV overlapped the 64 px instrument bubble and were read as 'FIC-10' or not at all: a fixture defect
which drawing / unit	0/3	two causes: the vision model never read the unit title from the title block (a real miss), and the scorer wanted 'P&ID-002' while the answer said 'PID-002' (same id without the ampersand)
Mean latency	33.8 s	document flow plus the longer drawing chunk

Changes before run 2, both recorded here and neither touching a prompt or the grader: (1) pid/make_pid.py instrument bubble radius 32 to 46 px (the fixture bug; the drawings are regenerated deterministically); (2) the golden 'which drawing' fact accepts P&ID-00k or PID-00k (a scorer equivalence, as doc-008 was). The unit-title fact stays, so that miss keeps counting.

Run 2 (bubbles 46 px; results/pid_eval_run2.log, results/pid_eval_summary.md):

Drawing	Tags expected	Correct	Invented	Missed
P&ID-001	13	11	-	T-101, T-102 (the tanks this time; a different image gives a different reading)
P&ID-002	13	12	-	FCV-201
P&ID-003	13	13	-	-

QA (run 2)	Run 1	Run 2
Routed to documents / right drawing retrieved	100% / 100%	100% / 100%
Answer correct overall	4/12	5/12
what an instrument measures, on what	3/3	3/3
valve isolating a pump	1/3	1/3 (grader: 'suction' not in the description)
controller + control valve	0/3	1/3 (tags now readable; the chained question still hard for the 3B model)
which drawing / unit	0/3	0/3 (the vision model does not read the title-block subtitle)
Mean latency	33.8 s	33.2 s

What the P&ID branch establishes: the ingestion path works end to end (a drawing is one more document; the right drawing is retrieved 24/24 times); a 3B vision model on CPU reads 87-92% of the tags and the valve/instrument relations correctly in ~100 s; what fails is the same grader that limits the text pipeline, plus title-block subtitles. For Kumar's real P&IDs: start from a golden set like pid/golden_pid.jsonl, expect small text near symbol edges to be the first failure, and run a tiled pass plus a stronger grader on a GPU or hosted model. Nothing in this branch was tuned after seeing results except the two recorded fixture/scorer changes.

Step 25. Showcase site and deployment (GitHub Pages live; Vercel prepared). The pipeline needs Postgres + pgvector and Ollama, so it cannot run on Vercel or Pages; what is deployable is a static site: site/index.html with the flow diagram, the results, the step phases, screenshots, the P&ID section, links to these PDFs, and a replay of 64 recorded runs (route, node timings, answer, and for version cases the documents retrieved OFF vs ON) from site/data/replay.json, which site/build_replay.py rebuilds from results/. The site has no backend; the replay is recorded, and says so. A 15-slide deck (docs/ragcap_deck.pptx, PDF alongside; built by docs/build_deck.js with pptxgenjs, validated and visually checked) tells the same story for a live audience: flow, phases, method, results, version filter, decisions, interfaces, P&ID, how to test, and the Vercel steps.

Where	How	Status
GitHub Pages	publish_site.sh: rebuilds replay.json, pushes site/ as the gh-pages branch, Pages serves it. Enabled once with gh api repos/.../pages (source gh-pages, path /).	live: https://shreenathacc22.github.io/ragcap/
Vercel	vercel.json at the repo root: outputDirectory 'site', no build command, cleanUrls. Import the GitHub repo once in the Vercel dashboard (steps below); every push to main redeploys.	prepared; needs a Vercel login (the CLI was not installed or logged in on this Mac, and a login is an interactive browser step)

Vercel, step by step (about 3 minutes): (1) vercel.com, sign in with GitHub. (2) Add New > Project > Import Git Repository > shreenathacc22/ragcap (authorise the Vercel GitHub app for that repo if asked). (3) Framework Preset: Other. Root Directory: leave '.' (vercel.json sets the output directory to site/). Build Command: leave empty. Output Directory: site. (4) Deploy. The first URL looks like ragcap-xxxx.vercel.app. (5) Settings > Domains > add ragcap.vercel.app (or any name that is free). (6) From now on, git push to main redeploys; the Deployments tab shows each build. CLI alternative from the repo root: npm i -g vercel; vercel login; vercel --prod. After deploying, paste the URL into README.md, site/index.html (links) and the deck, and re-run publish_site.sh.

```
./publish_site.sh                                # GitHub Pages: rebuild replay, push gh-pages
gh api repos/shreenathacc22/ragcap/pages --jq .html_url   # where Pages serves it
npm i -g vercel && vercel login && vercel --prod    # Vercel CLI path (interactive login once)
```

8. Pending / next

Everything in scope is closed (steps 1-25). What remains needs either your account or different hardware: (a) Vercel: import the repo once in the dashboard (step 25); GitHub Pages is already live. (b) LangSmith: set a real API key to see the uploads. (c) Stronger grader: run GRADE_MODE=escalate with an 8B+ model on a GPU or hosted endpoint; expected to fix the 3 deterministic held-out failures and the P&ID valve questions. (d) Real P&IDs: write the golden set first (pid/golden_pid.jsonl is the template), use a GPU-class vision model and a tiled pass for small text. (e) Docling on Apple Silicon or Linux for layout-aware PDF parsing. (f) Ideas not started: a validation retry on empty or implausible SQL results; multi-hop retrieval per sub-question.